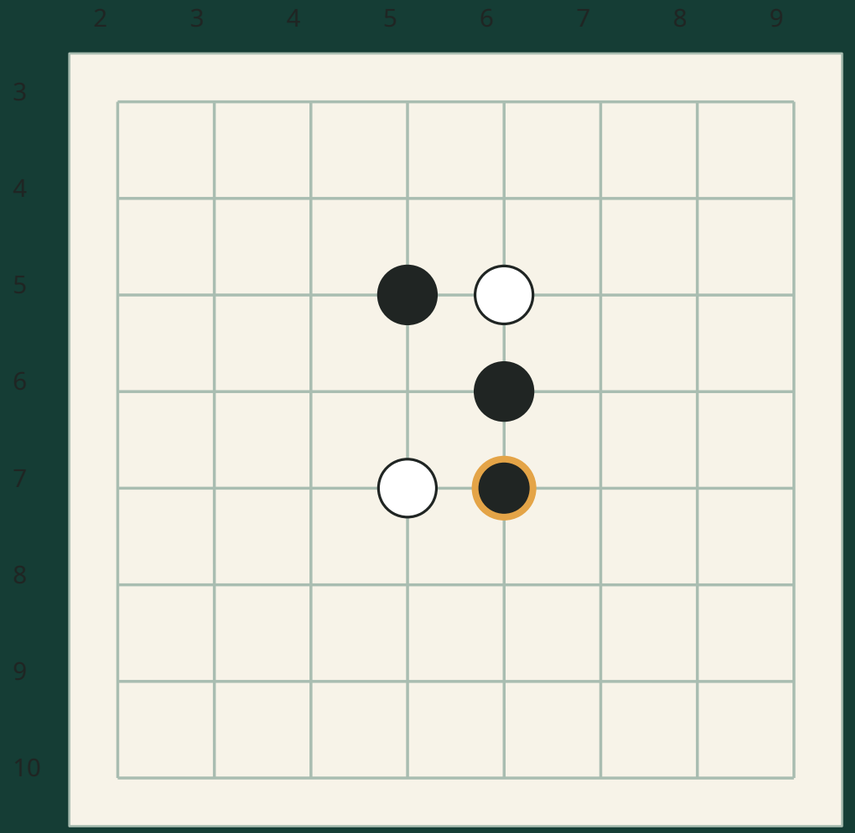
Caro AI — Weeks 03–07

Progress Report · Data Structures, Algorithms and Recursive Search
From a playable 15 × 15 board to verified native search experiments.
Source review: 21 October 2026 · Experiment snapshot: 17 October 2026.

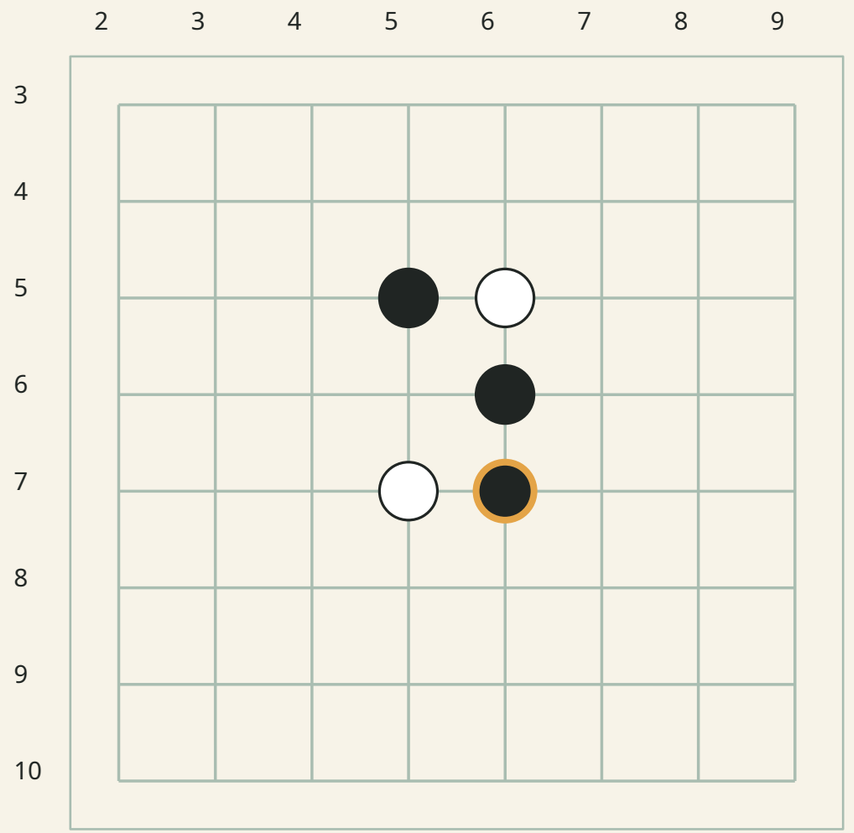


Weeks 03–07 | Scope and evidence

03 Interface and lifecycle · 04 Patterns and heuristics · 05 Minimax · 06 Alpha-Beta and cache · 07 Spatial candidates.
Current app: React/PixiJS → tactical guard → Worker → existing Gomoku Wasm.
Research: separate native caro-core-reference library.
Native results do not prove shipped-Wasm equivalence. Week labels organize reports; they are not commit dates.

Week 03 | What happens after one tap?

1 Convert screen pixels to a board cell.
2 Check pause, round, turn and AI-thinking gates.
3 Reject occupied or out-of-bounds cells.
4 Commit the stone and append history.
5 Check win first, then draw, otherwise switch player.
6 Publish a snapshot to React and PixiJS.
This move transaction is iterative.



Which data structures own the move?

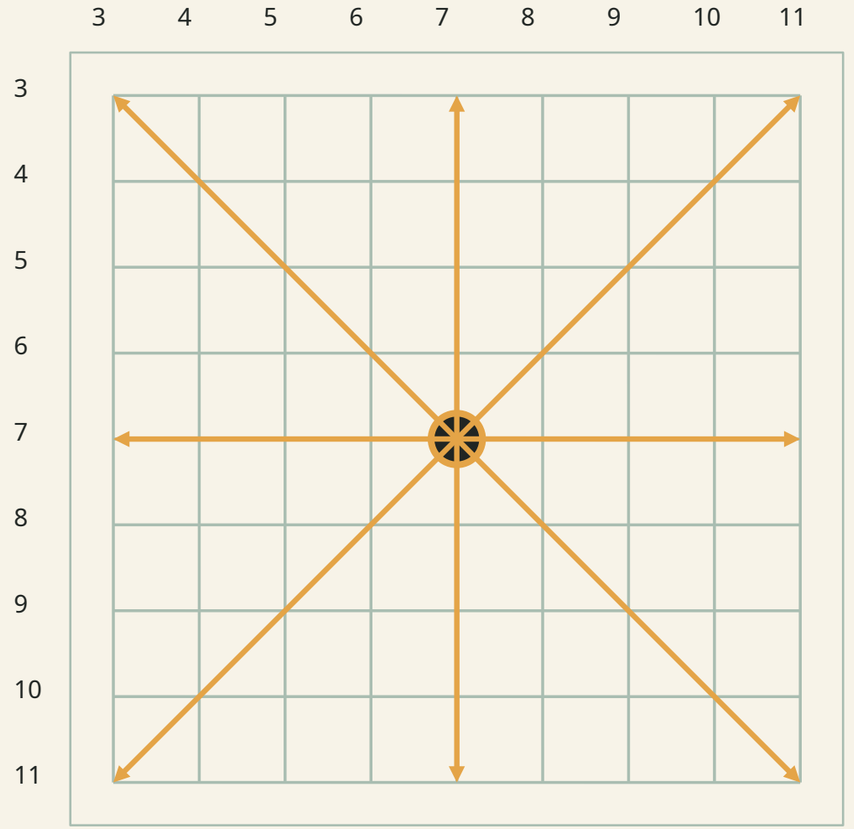
Live TypeScript board — Map<string, Player>, keyed by row,col.
Move history — Move[], preserving turn order.
Change subscribers — Set of listeners.
Input and round phases — finite state machines.
Native reference board — [i8; 225], indexed by row × 15 + column.
These are different representations. They require an explicit adapter contract before integration.

Pixels become coordinates before rules run

column = floor((screenX − cameraX) / zoom / 56)
row = floor((screenY − cameraY) / zoom / 56)
Input states distinguish a tap from a camera drag.
BoardBounds then enforces rows and columns 0–14.
Coordinate conventions must be normalized: UI (row, column), Rust API (x, y).

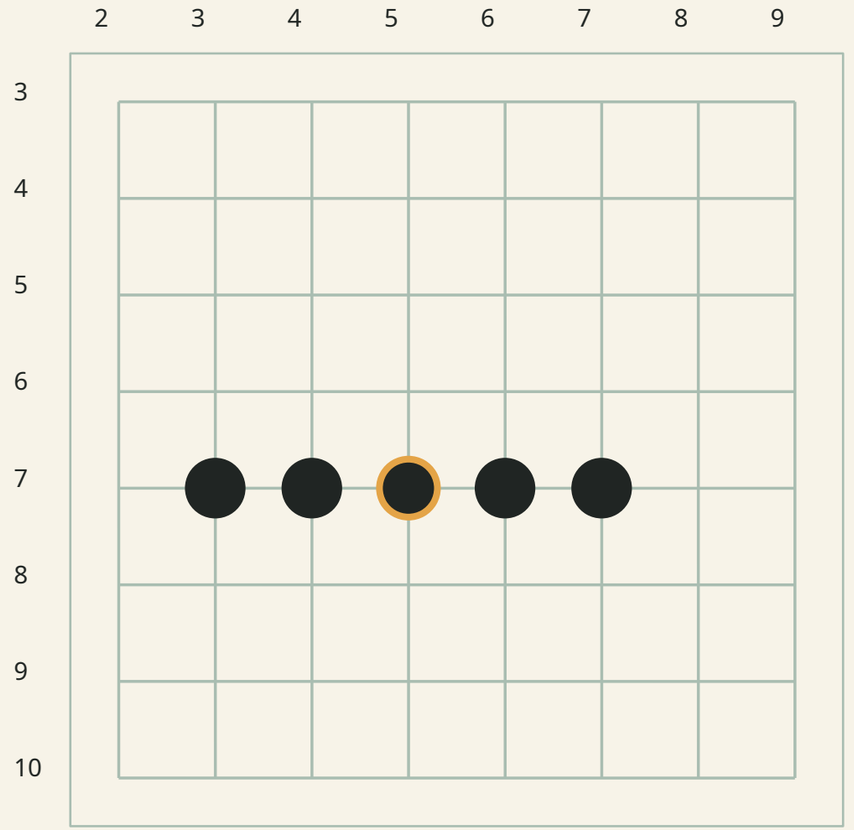
The win scan uses four axes

Directions: (0,1), (1,0), (1,1), (1,−1).
For each axis: L = 1 + forward matches + backward matches.
Current Freestyle rule: win when any L ≥ 5.
The live scanner checks at most four neighbors per ray: 4 × 2 × 4 = 32 neighbor probes.
Loops perform the scan; recursion is unnecessary here.



Worked scan | Why the last move is enough

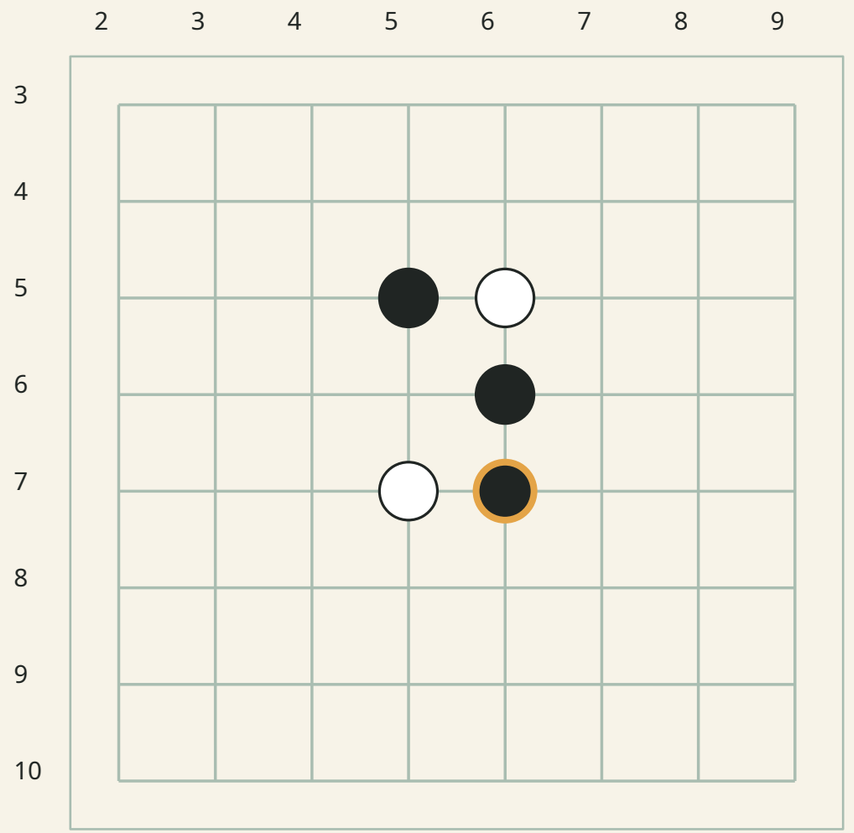
Place X at (7,5).
Matching X stones: (7,3), (7,4), (7,6), (7,7).
Horizontal count: 1 + 2 left + 2 right = 5 → WIN.
Only lines crossing the new stone can become newly winning after a legal nonterminal state.
Blocked endpoints do not invalidate this Freestyle win.



2 left + 1 + 2 right = 5

An AI turn starts with a tactical guard

Scan empty cells for an immediate own win; otherwise find an immediate opponent win to block.
If neither exists, send move history to the Worker.
The Worker replays history into Rust Game and calls ai_move(difficulty).
Map (x,y) to (row=y,column=x), reject stale results, revalidate, then commit.
One selected block cannot resolve every double threat.



Week 03 progress | Playable structure is present

Source reviewed: 15 × 15 board; React/PixiJS responsibilities; tap/drag handling; move gates; round transitions; Worker request IDs and stale-result protection.
Cancellation terminates and recreates the Worker.
Pending runtime evidence: browser interaction, device latency and frame rate.
Frontend test rerun was blocked by missing Vitest dependencies; older FPS claims are not reused.

Week 04 | A heuristic scan counts patterns

The reference scans board cells for one player across four axes.
Start only where the preceding cell is not the same player.
Walk the maximal contiguous run; count empty endpoints.
For runs of at least two stones, emit one Pattern record, then increment PatternCounts.
First-stone ownership prevents recounting the same run along one axis.

. X X X .

Start only at first X.

length = 3
open_ends = 2

Emit one maximal run.

Week 04 | Pattern weights and position score

Native reference weights: five+ 1,000,000; open four 100,000; closed four 20,000; open three 5,000; closed three 500; open two 100; closed two 10.
E(s)=Attack−Defense; default β=1. One own open three against one opponent closed three: 5,000−500=4,500, assuming no other patterns.
Rules determine terminal wins. Game-engine weights differ. XX.XX needs gap-aware detection; β=1.15 remains proposed.

Pattern	Example	Weight
Five+	XXXXX	1,000,000
Open four	.XXXX.	100,000
Closed four	OXXXX.	20,000
Open three	.XXX.	5,000
Closed three	OXXX.	500
Open two	.XX.	100
Closed two	OXX.	10

A board edge is not an open endpoint.

Three baselines isolate algorithm behavior

Random: enumerate all legal moves; choose using seeded SplitMix64.
Rule-based: try own win → block opponent win → heuristic fallback.
Heuristic: for each generated candidate, play → evaluate → undo; keep the first maximum.
The heuristic baseline is greedy one-ply evaluation. It does not recursively inspect opponent replies.

Week 04 progress | Evaluation exists; migration needs proof

Present in the native reference: contiguous patterns, default weights, terminal handling and reproducible baselines.
The reference crate is rlib-only and has no wasm-bindgen exports.
Existing game Wasm and the native reference are separate artifacts.
Pending: shared TS/Rust fixtures, adapter verification, broken-threat tests and controlled browser benchmarks.
No TS-to-Wasm speedup is claimed.

Week 05 | Recursive Minimax

Scores stay in root-player perspective: MAX on our turn, MIN on the opponent's.

```
SEARCH(s,d):
  if terminal: return utility
  if d=0: return heuristic
  generate moves; if empty: return heuristic
  for each move:
    token=play(move)
    child=SEARCH(s,d−1)
    undo(token)
  return MAX or MIN of children
```

One ply is one move; depth 2 includes an opponent reply.

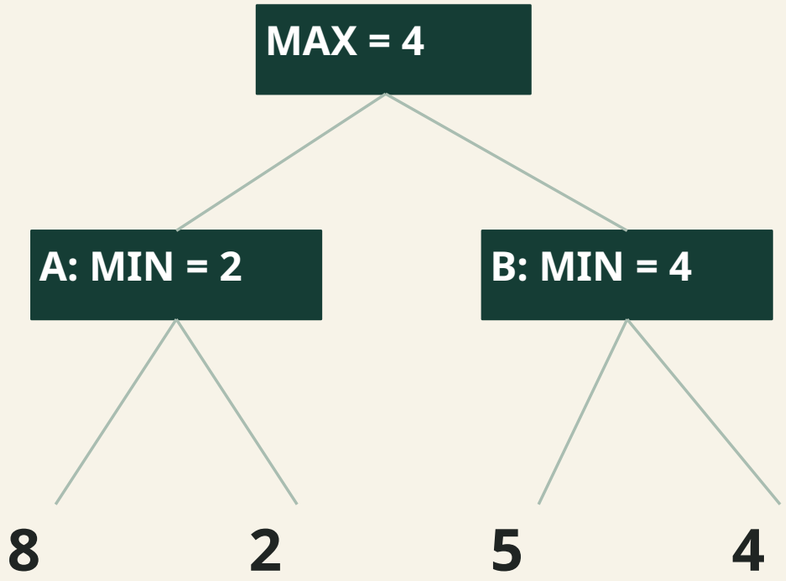
BASE CASE
terminal or depth = 0

RECURSIVE STEP
play → SEARCH(d−1) → undo

MAX = maximize
MIN = minimize

Worked recursion | Protect against the best reply

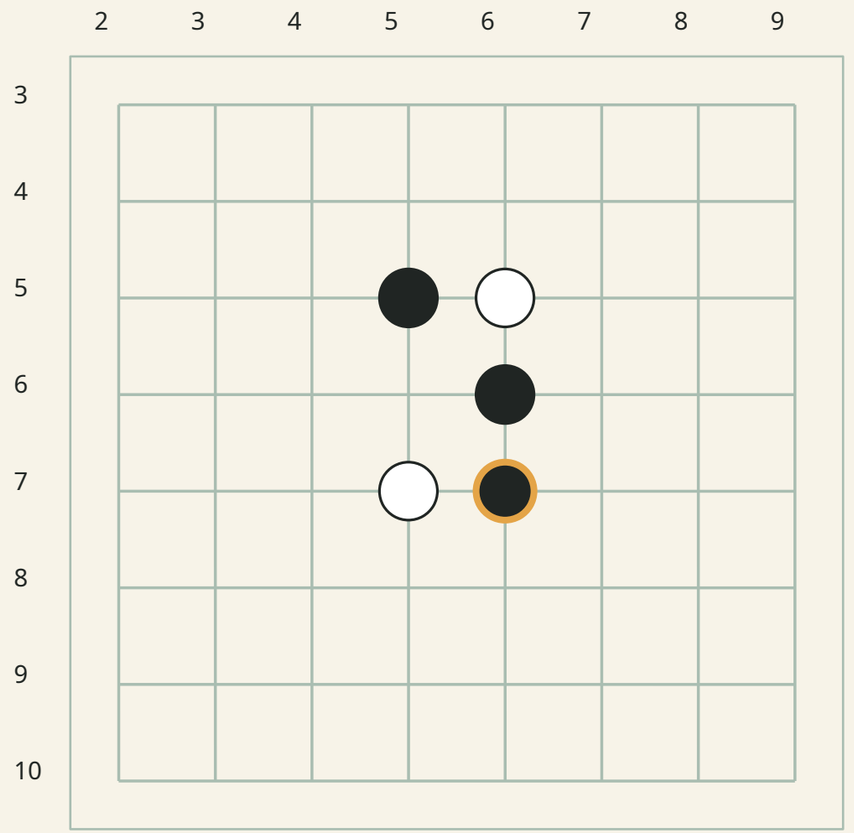
Illustrative tree, not an engine measurement.
Move A allows replies scored 8 and 2: A = min(8,2) = 2.
Move B allows replies scored 5 and 4: B = min(5,4) = 4.
Root chooses max(2,4) = 4, therefore B.
Choosing the largest leaf alone would ignore the opponent's choice.



min(8,2)=2
min(5,4)=4
max(2,4)=4 → B

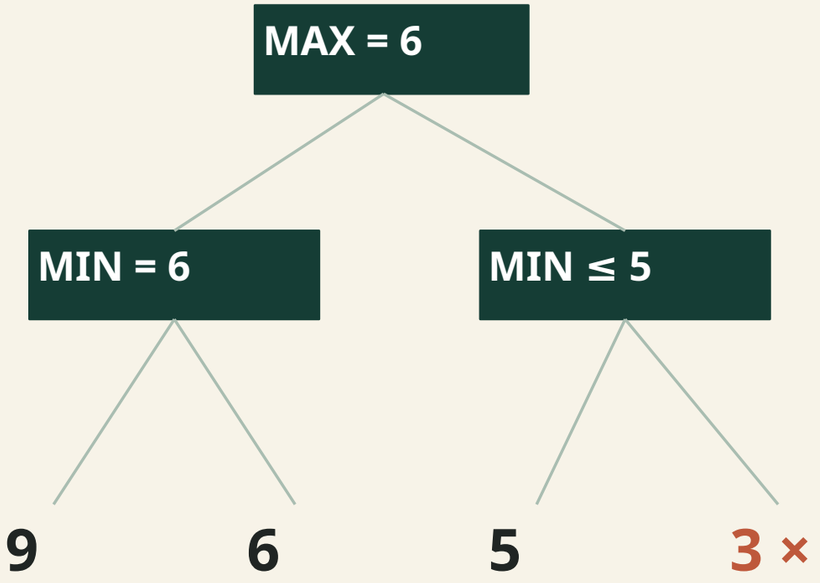
Week 05 progress | Reversible search and terminal scores

UndoToken restores cell, player, status, last move and move count, so every sibling starts from the same parent.
Utility: win M−ply; loss −M+ply; draw 0; M=10,000,000.
Depth limit uses the heuristic. Uniform branching b and depth d gives O(b^d) traversal; call stack O(d) plus retained move vectors.
Recorded: 551 root restorations and 112 Minimax/Alpha-Beta matching score-and-move pairs.



Week 06 | Alpha-Beta cutoff example

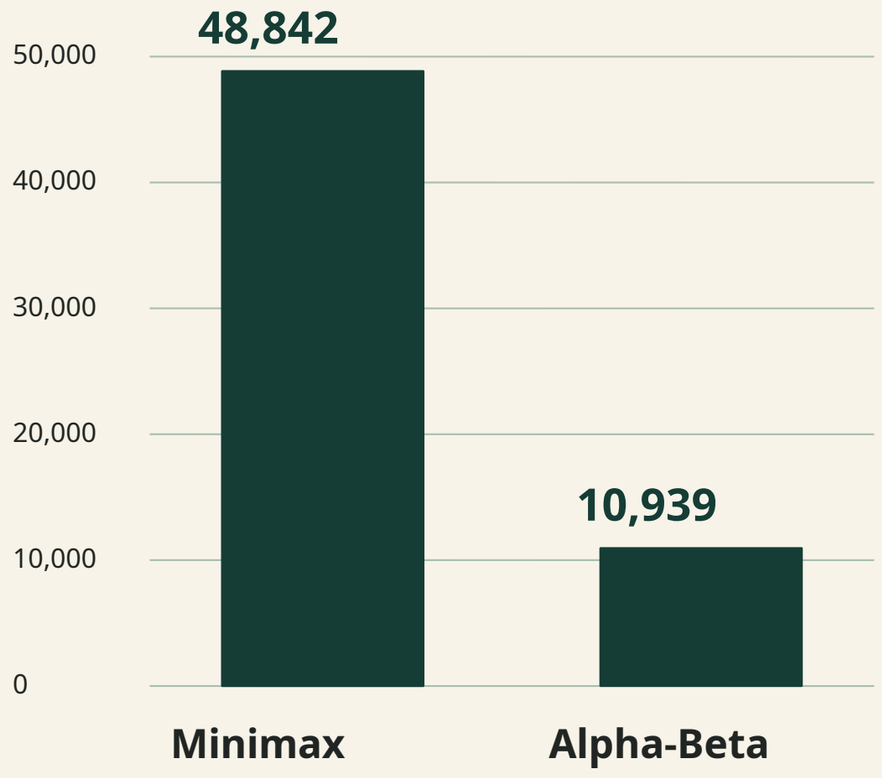
At MAX update alpha; at MIN update beta. Undo each move before skipping siblings when alpha≥beta.
Illustrative tree: left MIN sees 9,6 → 6, so alpha=6. Right MIN sees 5, so beta=5; skip its final leaf 3.
MAX remains 6. Full traversal: 7 nodes; pruned: 6.
The same finite tree retains its Minimax value; omitted candidates are a separate issue.
Alpha: best MAX alternative. Beta: best MIN alternative.



α = 6 ≥ β = 5
Last leaf is skipped.

Measured pruning | Same answer, fewer nodes

Clustered fixture · all legal moves · depth 2 · natural order.
Minimax: 48,842 nodes.
Alpha-Beta: 10,939 nodes.
Node reduction: 77.60%.
Both: score 0; move (6,7).
Native Rust release results, 17 October 2026. This is one controlled configuration, not browser performance.



Visited search nodes

search_summary.csv rows 52–53 · provenance.json

Week 06 progress | Cache bounds and an open hazard

Present: bound updates, cutoffs and TTEntry {depth,score,Exact/Lower/Upper,best_move} in HashMap<u64,TTEntry>.
Zobrist key covers pieces and side to move.
Recorded after changed root: reused table score 19,800; fresh table 0. Keep a fresh compatible table until context is repaired.
208 cold-table score comparisons passed in the wider native package. Fewer nodes do not guarantee lower elapsed time.

REUSED TT

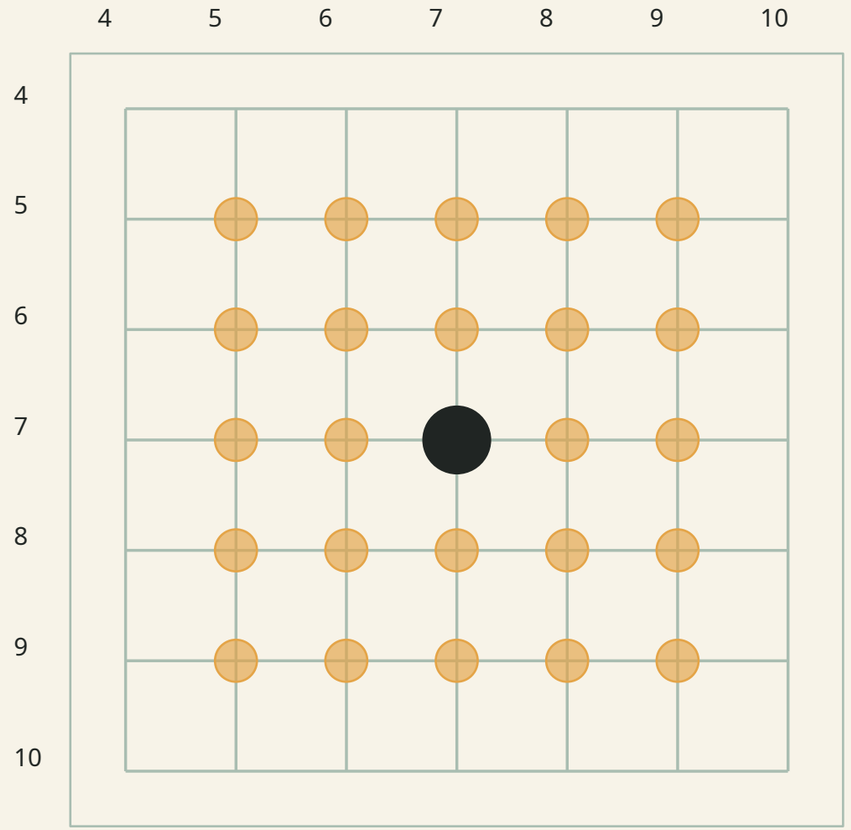
19,800

FRESH TT

0

Week 07 | Nearby empty cells become candidates

Chebyshev distance: d∞ = max(|Δrow|, |Δcolumn|).
Keep empty cells within radius R of at least one stone.
Reference defaults: R = 2; no cap.
With one center stone: 224 legal cells, 24 radius-2 candidates.
Empty board special case: center (7,7).
Spatial filtering reduces branching without proving every excluded move irrelevant.



R = 2 → 24 candidates

The candidate scan uses marking and a vector

1 Scan the flat board for occupied cells.
2 Visit each square neighborhood.
3 Reject occupied and out-of-board cells.
4 Mark [bool;225] to deduplicate overlaps.
5 Scan marks row-major into Vec<Move>.
6 Apply optional truncation, then optional move ordering.
Generation changes membership; ordering changes sequence.

[bool; 225]
Mark membership once.

↓ scan indices

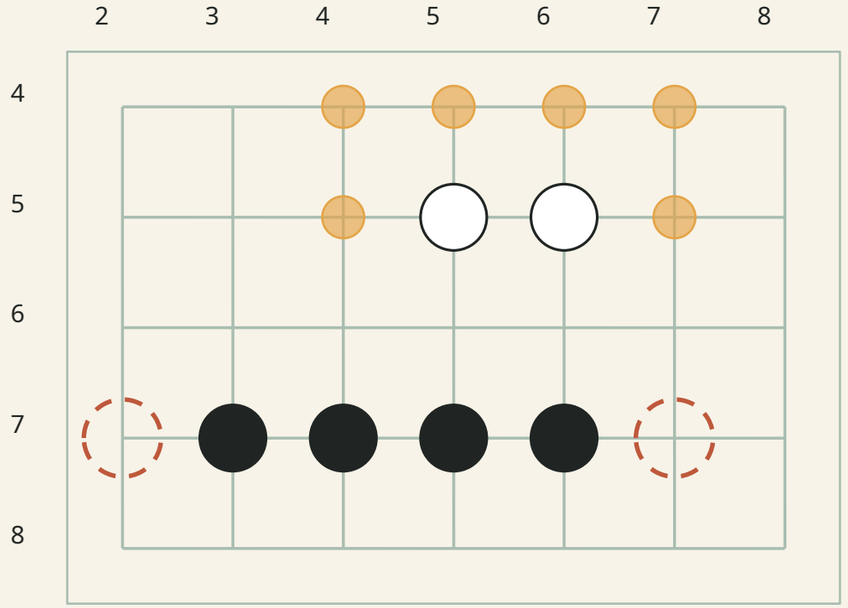
Vec<Move>
Deterministic row-major order.

↓ optional cap

Search surviving moves.

A candidate cap can delete a winning move

Recorded win fixture: radius 1 produces 20 candidates.
Cap 6 retains the first six in row-major order.
Both winning endpoints (7,2) and (7,7) disappear.
Depth-1 score changes: 9,999,999 → 99,990.
Sorting the surviving moves cannot recover a deleted move.
Tactical preservation must be tested before adding a cap.



Zero-based (row, column)
Amber: first six candidates
Dashed: omitted winning endpoints

Week 07 progress | Geometry verified, coverage remains open

Recorded: 48 geometry configurations passed uniqueness, emptiness and radius checks.
Source review confirms deterministic row-major output; edge and cap examples are recorded.
One center stone: R1=8, R2=24, R3=48.
One corner stone: R1=3, R2=8, R3=15.
Candidate time: $O(N^2 + kR^2)$; $O(N^2)$ marking/output space.
N=board width; k=occupied cells. Cap applies after generation.

Stone position	R1	R2	R3
Center	8	24	48
Corner	3	8	15

Radius expands a square (Chebyshev neighborhood).

Every scan has a different purpose

Win check — last move, four bounded axes; iterative.
Tactical guard — empty-cell enumeration plus hypothetical win checks; iterative.
Pattern evaluation — whole-board maximal-run scan; iterative, $O(N^2)$ per player.
Candidate generation — neighborhood marking; iterative, $O(N^2+kR^2)$.
Minimax/Alpha-Beta — depth-first recursion over generated moves, with make/undo.
The call stack is O(d); retained move vectors also consume memory.

Progress is supported by bounded evidence

Today: 6 unit + 3 integration reference tests passed.
Stored 17 October package: 20 fixture positions; 112 Minimax/Alpha-Beta pairs; 551 root restorations; 48 candidate sets.
Current 26-file source hashes match that package.
Next: preserve tactical moves, repair TT context, add broken-threat fixtures, verify the adapter/Worker in browsers, then measure latency and playing strength.
Frontend runtime remains unverified.

Sources and interpretation

Implementation: tducn110/caro, reviewed checkout c1e7447; core GameState, WinRules, GameController, AIController and Worker.
Research: Week_03–Week_07 reports; caro-core-reference; research/week_05_09/results.
Theory: UC Berkeley CS188, Minimax and Alpha-Beta.
Data structures: official Rust collections documentation.
Results are native reference evidence, not device, browser, tournament or production acceptance.

UC Berkeley CS188 — Minimax and Alpha-Beta

https://inst.eecs.berkeley.edu/~cs188/textbook/games/minimax.html

Official Rust collections documentation